# CORE JAVA NOTES

## Overview of the topics:

**1. Java Basics & Fundamentals**

- **Java Architecture:** JVM, JRE, and JDK differences.
- **Variables & Data Types:** Primitives (int, double) and references.
- **Operators:** Arithmetic, logical, relational, and bitwise.
- **Control Statements:** If-else conditions and switch-case blocks.
- **Loops:** For, while, and do-while loops.
- **Arrays:** Single and multi-dimensional array manipulation.

**2. Object-Oriented Programming (OOP)**

- **Classes & Objects:** Blueprints and state instantiation.
- **Methods:** Parameters, return types, and overloading.
- **Constructors:** Default, parameterized, and constructor chaining.
- **Inheritance:** Single, multilevel, and hierarchical structures.
- **Polymorphism:** Method overriding and dynamic method dispatch.
- **Abstraction:** Abstract classes versus interface designs.
- **Encapsulation:** Access modifiers (private, protected, public).
- **Keywords:** Proper usage of `this`, `super`, `static`, and `final`.

**3. Core Built-in Classes**

- **String Handling:** String, StringBuilder, and StringBuffer classes.
- **Wrapper Classes:** Autoboxing and unboxing primitive types.
- **Math & Object:** Methods inside the base `Object` class.

**4. Exception Handling**

- **Try-Catch Blocks:** Handling runtime errors gracefully.
- **Hierarchy:** Checked exceptions versus unchecked exceptions.
- **Keywords:** Using `throw`, `throws`, and `finally`.
- **Custom Exceptions:** Creating user-defined exception classes.
- **Try-with-resources:** Automatic management of closable resources.

**5. Java Collection Framework**

- **List Interface:** ArrayList, LinkedList, and Vector usage.
- **Set Interface:** HashSet, LinkedHashSet, and TreeSet mechanics.
- **Map Interface:** HashMap, LinkedHashMap, and TreeMap operations.
- **Queue Interface:** PriorityQueue and Deque implementations.
- **Sorting:** Implementing `Comparable` and `Comparator` interfaces.

**6. Java I/O & File Handling**

- **Byte Streams:** FileInputStream and FileOutputStream classes.
- **Character Streams:** FileReader and FileWriter classes.
- **Buffering:** BufferedReader and BufferedWriter for efficiency.
- **Serialization:** Converting objects into byte streams.

**7. Multithreading & Concurrency**

- **Thread Creation:** Extending `Thread` versus implementing `Runnable`.
- **Lifecycle:** Thread states from New to Terminated.
- **Synchronization:** Preventing data corruption with synchronized blocks.
- **Inter-thread Communication:** Using `wait()`, `notify()`, and `notifyAll()`.

**8. Advanced Core Features (Java 8+)**

- **Lambda Expressions:** Writing clean, functional code blocks.
- **Functional Interfaces:** Predicate, Consumer, Supplier, and Function.
- **Stream API:** Filtering, mapping, and reducing collections.
- **Optional Class:** Handling null values without crashes.